

Resolución del juego *Lights Out* mediante sistemas de ecuaciones lineales con aritmética binaria

Guillermo Rivero (C.I. 5048260-2)
Enzo Barreto (C.I. 4523297-3)

Facultad de Ingeniería y Tecnologías, Universidad Católica del Uruguay

Álgebra Aplicada

14 de octubre de 2026

Resumen

Lights Out es un juego que se juega en un tablero de $n \times n$ luces, en el que presionar una luz cambia su estado y el de sus vecinas ortogonales; la meta es apagarlas todas. El objetivo de este trabajo es modelar las soluciones del juego como un sistema de ecuaciones lineales sobre el conjunto $\{0,1\}$ con suma binaria, e implementar en Python un método que lo resuelva. Primero se demuestra que ninguna luz necesita presionarse más de una vez y que el orden de las pulsaciones no altera el resultado, lo que permite representar cada solución con un vector de n^2 componentes binarias. Luego se construye el sistema $A\mathbf{x} = \mathbf{b}$, se justifica cada ecuación del ejemplo 3×3 y se implementa la escalerización de Gauss-Jordan con suma binaria. La implementación se validó con los ejemplos dados y con 20 000 tableros aleatorios para n entre 1 y 10, comprobando por simulación cada solución obtenida y comparando con una búsqueda exhaustiva en tableros pequeños. Los resultados muestran que la matriz del sistema es invertible para $n \in \{1,2,3,6,7,8,10\}$, pero no para $n \in \{4,5,9\}$, en cuyo caso solo una fracción 2^{-d} de los tableros tiene solución, siendo d la dimensión del núcleo; las frecuencias observadas coinciden con ese valor. Además, se encontró que, si se toma cada fila tal como está, la entrada en la posición de pivote se anula en el segundo paso para todo $n \geq 2$; el problema se resuelve sumando a esa fila otra inferior, sin salir de la transformación $F_i \to F_i + F_j$. Se concluye que el modelo lineal resuelve el juego de forma exacta y permite decidir de antemano si un tablero tiene solución.

Palabras clave: Lights Out, sistemas lineales, aritmética binaria, escalerización gaussiana, núcleo.

1. Introducción

Lights Out es un juego electrónico que se juega sobre un tablero cuadrado de $n \times n$ luces, cada una prendida o apagada. El juego parte de una configuración cualquiera y el jugador busca llegar al estado en que todas las luces están apagadas. La dificultad radica en que presionar una luz no solo cambia su estado, sino también el de las luces adyacentes a ella (arriba, abajo, a la izquierda y a la derecha), por lo que cada jugada puede deshacer parte del progreso logrado con las anteriores. Para tableros pequeños es posible encontrar una solución por ensayo y error, pero el número de combinaciones de pulsaciones posibles crece como 2^{n^2}: ya en un tablero de 5×5 hay más de 33 millones, lo que vuelve impracticable la búsqueda exhaustiva.

El interés del problema reside en que, a pesar de su apariencia combinatoria, tiene una estructura lineal. Si se identifica una luz prendida con el número 1, una apagada con el 0 y se suma con la regla $1+1=0$, el efecto de presionar un conjunto de luces es una suma, y encontrar una solución equivale a resolver un sistema de ecuaciones lineales (ver Anderson y Feil, 1998). Esto permite aplicar las herramientas del curso, en particular la escalerización gaussiana, a un problema concreto y verificar los resultados de manera computacional. El problema también obliga a revisar qué propiedades de la escalerización dependen de los números reales y cuáles se mantienen cuando se trabaja solo con ceros y unos.

El objetivo general de este trabajo es resolver el juego *Lights Out* en tableros de $n \times n$ mediante un sistema de ecuaciones lineales con suma binaria. Los objetivos específicos son los siguientes:

(a) justificar que una solución del juego puede representarse con un vector de n^2 componentes en $\{0,1\}$, mostrando que ninguna luz necesita presionarse más de una vez y que el orden de las pulsaciones no influye en el resultado;

(b) construir y justificar el sistema de ecuaciones asociado a un tablero, detallando el caso 3×3 propuesto en la consigna y comprobando la solución dada para ese caso;

(c) implementar en Python una función que reciba el estado inicial del tablero y devuelva una solución, adaptando la escalerización gaussiana a la suma binaria;

(d) analizar para qué tamaños de tablero todo estado tiene solución y qué ocurre cuando no la tiene.

Para cumplir estos objetivos se combinan dos enfoques. Por un lado, se demuestran las propiedades del juego que sustentan el modelo. Por otro, se implementa el método de resolución y se lo evalúa con los ejemplos de la consigna, con una captura del juego en línea y con tableros aleatorios, verificando cada solución mediante una simulación del juego independiente del álgebra.

El informe se organiza de la siguiente manera. En la Sección 2 se presentan las reglas del juego, la aritmética binaria y los resultados sobre sistemas lineales que se utilizan. En la Sección 3 se desarrolla el modelo, se construye el sistema y se describen el algoritmo, su

implementación y el diseño de las pruebas. La Sección 4 presenta los resultados obtenidos, que se interpretan en la Sección 5. Finalmente, la Sección 6 resume las conclusiones del trabajo.

2. Marco teórico

2.1. Reglas del juego y representación del tablero

El tablero se representa con una matriz $T = (a_{ij})$ de tamaño $n \times n$, donde $a_{ij} = 1$ si la luz de la fila i y la columna j está prendida y $a_{ij} = 0$ si está apagada (Universidad Católica del Uruguay, s.f.). Dos luces son *adyacentes* si comparten un lado, es decir, si están en la misma fila y en columnas consecutivas, o en la misma columna y en filas consecutivas. Para cada luz p se define su *vecindario* $V(p)$ como el conjunto formado por p y sus luces adyacentes. Si $n \geq 3$, una luz en el interior del tablero tiene un vecindario de cinco luces, una luz en un borde (que no sea esquina) tiene cuatro y una luz en una esquina tiene tres.

La regla del juego es que presionar la luz p cambia el estado de todas las luces de $V(p)$ y deja las demás sin cambios. Como la relación de adyacencia es simétrica, $q \in V(p)$ si y solo si $p \in V(q)$: presionar p afecta a q exactamente cuando presionar q afecta a p.

2.2. Aritmética binaria

Se trabaja en el conjunto $\mathbb{F}_2 = \{0, 1\}$ con la suma y el producto definidos por

$$0+0=0, \quad 0+1=1+0=1, \quad 1+1=0, \qquad 0\cdot 0=0\cdot 1=1\cdot 0=0, \quad 1\cdot 1=1.$$

Estas operaciones corresponden a calcular el resto de la división entre 2 del resultado usual, y $\mathbb{F}_2$ con ellas es un cuerpo: la suma y el producto son conmutativos y asociativos, el producto es distributivo respecto de la suma, el 0 y el 1 son neutros, y el único elemento no nulo, el 1, es su propio inverso multiplicativo (Lidl y Niederreiter, 1997). Dos propiedades se usan de forma continua en este trabajo. La primera es que todo elemento es su propio opuesto, $x + x = 0$, de modo que restar es lo mismo que sumar. La segunda es que cambiar el estado de una luz equivale a sumarle 1: si estaba apagada pasa a $0 + 1 = 1$ y si estaba prendida pasa a $1 + 1 = 0$.

Como $\mathbb{F}_2$ es un cuerpo, los resultados de álgebra lineal que se demuestran usando solo las propiedades de cuerpo (escalerización, rango, núcleo, teorema de las dimensiones) siguen valiendo al reemplazar los números reales por $\mathbb{F}_2$, ya que en esos textos se desarrollan sobre un cuerpo cualquiera (Hoffman y Kunze, 1971). En lo que sigue, $\mathbb{F}_2^m$ denota el conjunto de vectores de m componentes en $\mathbb{F}_2$.

2.3. Sistemas lineales, escalerización y núcleo

Un sistema de m ecuaciones lineales con m incógnitas se escribe como $A\mathbf{x} = \mathbf{b}$, con A una matriz $m \times m$ y $\mathbf{b} \in \mathbb{F}_2^m$. La *escalerización gaussiana* transforma la matriz ampliada $[A \,|\, \mathbf{b}]$ mediante transformaciones elementales (intercambiar dos filas, multiplicar una fila por un escalar no nulo y sumar a una fila un múltiplo de otra) hasta obtener una forma escalerizada,

sin cambiar el conjunto de soluciones (Lay et al., 2016). Si además se anulan las entradas por encima de cada pivote, se obtiene la forma escalerizada reducida (método de Gauss-Jordan).

En $\mathbb{F}_2$ la escalerización se simplifica: el único escalar no nulo es el 1, de modo que multiplicar filas no tiene efecto, y sumar a F_i un múltiplo no nulo de F_j es simplemente $F_i \to F_i + F_j$. Cuando la entrada en la posición de pivote es 0, hay que conseguir un 1 a partir de alguna fila inferior que tenga un 1 en esa columna. Esto puede hacerse intercambiando las filas, pero también sumando la fila inferior a la del pivote, que es otra transformación del tipo $F_i \to F_i + F_j$; por eso, en $\mathbb{F}_2$, esa suma de filas alcanza para escalerizar cualquier matriz. Si ninguna fila inferior tiene un 1 en la columna, esta no tiene pivote y su incógnita es *libre*.

El *rango* r de A es la cantidad de pivotes. El sistema es compatible si y solo si la escalerización no produce una fila de la forma $0 = 1$ (teorema de Rouché-Frobenius). El *núcleo* de A es $N(A) = \{\mathbf{x} \in \mathbb{F}_2^m : A\mathbf{x} = \mathbf{0}\}$, y por el teorema de las dimensiones $\dim N(A) = m - r$. Si $\mathbf{x}_p$ es una solución del sistema, el conjunto de todas sus soluciones es $\{\mathbf{x}_p + \mathbf{z} : \mathbf{z} \in N(A)\}$ (Hoffman y Kunze, 1971; Lay et al., 2016).

A diferencia de lo que ocurre con los números reales, en $\mathbb{F}_2$ los conjuntos son finitos. Un subespacio de dimensión d tiene exactamente 2^d elementos, porque cada vector se escribe de forma única como combinación de d vectores de una base con coeficientes en $\{0, 1\}$. En consecuencia, un sistema compatible tiene exactamente $2^{\dim N(A)}$ soluciones, y la imagen de A, que es el conjunto de vectores $\mathbf{b}$ para los que el sistema es compatible, tiene 2^r elementos de los 2^m posibles.

2.4. Antecedentes

El enfoque lineal del juego es conocido. Anderson y Feil (1998) plantean el mismo sistema para el tablero de 5×5, muestran que su matriz tiene rango 23 y caracterizan los tableros con solución mediante los vectores del núcleo. Sutner (1989) estudia el mismo problema desde la teoría de los autómatas celulares lineales y demuestra que el tablero con todas las luces prendidas siempre puede apagarse, sea cual sea su tamaño. Este trabajo parte de esas ideas y las aplica de forma sistemática a tableros de n entre 1 y 10, con una implementación propia de la escalerización.

3. Metodología

El trabajo se desarrolló en tres etapas: primero se demostraron las propiedades del juego que justifican el modelo vectorial; luego se construyó el sistema de ecuaciones y se diseñó el algoritmo de resolución; por último, se implementó el algoritmo y se lo evaluó con un conjunto de pruebas.

3.1. Modelo de las soluciones

Una *jugada* es una sucesión finita de pulsaciones $p_1, p_2, \ldots, p_k$, y *gana* el juego si al aplicarla al tablero inicial todas las luces quedan apagadas. Para cada luz p se llama c_p a la can-

tidad de veces que aparece en la sucesión. El efecto de una jugada se describe con la siguiente proposición.

Proposición 1. *Al aplicar una jugada al tablero inicial, el estado final de la luz q es*

$$a_q^{final} = a_q + \sum_{p\,:\,q\in V(p)} c_p \pmod 2. \tag{1}$$

Demostración. Cada pulsación de una luz p con $q \in V(p)$ suma 1 al estado de q, y las pulsaciones de luces p con $q \notin V(p)$ no lo modifican. Esto vale sea cual sea el estado del tablero en el momento de presionar, porque la regla del juego no depende de los estados actuales. Por lo tanto, el estado final de q es su estado inicial más la cantidad total de pulsaciones que lo afectan, calculada en $\mathbb{F}_2$, es decir, módulo 2. ■

De la Proposición 1 se deducen las respuestas a las dos primeras preguntas planteadas en la consigna.

Proposición 2 (No es necesario presionar una luz más de una vez)**.** *Si una jugada gana el juego, la jugada que presiona una sola vez cada luz con c_p impar, y ninguna otra, también lo gana.*

Demostración. En la Ecuación (1) solo interviene el resto de dividir cada c_p entre 2. Al reemplazar cada c_p por c_p mód $2 \in \{0,1\}$ el estado final de todas las luces no cambia, y por lo tanto sigue siendo el tablero apagado. En términos del juego: presionar dos veces una misma luz cambia dos veces el estado de cada luz de su vecindario y deja el tablero como estaba, de modo que las pulsaciones pueden eliminarse por pares. ■

Proposición 3 (El orden de las pulsaciones no influye)**.** *Dos jugadas que presionan cada luz la misma cantidad de veces, en cualquier orden, producen el mismo tablero final.*

Demostración. La Ecuación (1) depende solo de las cantidades c_p y no del orden de la sucesión, ya que la suma en $\mathbb{F}_2$ es conmutativa y asociativa. ■

Por ejemplo, en el tablero 3×3 de la consigna, presionar primero a_{33} y después a_{22} produce el mismo tablero que presionar primero a_{22} y después a_{33}: en ambos casos se suma 1 a las luces de $V(a_{22})$ y a las de $V(a_{33})$, y las luces a_{23} y a_{32}, que pertenecen a los dos vecindarios, reciben dos cambios y quedan como estaban.

Las Proposiciones 2 y 3 justifican el modelo propuesto en la consigna. Como el orden no importa, una solución queda determinada por las cantidades c_p; como basta presionar cada luz a lo sumo una vez, se puede suponer $c_p \in \{0,1\}$. Una solución es entonces un *conjunto* de luces a presionar, que se representa con un vector $\mathbf{x} = (x_1, \ldots, x_{n^2}) \in \mathbb{F}_2^{n^2}$.

Resta demostrar que la regla que asigna cada componente x_i a una luz es una correspondencia biunívoca entre los índices $\{1, \ldots, n^2\}$ y las posiciones del tablero. Dado i, el algoritmo de la división entera da un único par (q,r) con $i-1 = nq+r$ y $0 \le r < n$. Como

$0 \le i-1 \le n^2-1$, se cumple $0 \le q \le n-1$, por lo que $(q+1, r+1)$ es una posición válida. Recíprocamente, la luz a_{fc} corresponde al índice $i = n(f-1)+c$, de modo que la correspondencia es invertible. En otras palabras, las luces se numeran recorriendo el tablero por filas: $x_1, \ldots, x_n$ son las luces de la primera fila, $x_{n+1}, \ldots, x_{2n}$ las de la segunda, y así sucesivamente. Por lo tanto, cada vector de $\mathbb{F}_2^{n^2}$ representa exactamente un conjunto de pulsaciones y cada conjunto de pulsaciones está representado por exactamente un vector, lo que hace correcto el modelo.

3.2. Construcción del sistema de ecuaciones

Con el modelo anterior, la Ecuación (1) para la luz q que corresponde al índice i se escribe $a_q + \sum_{j : q \in V(p_j)} x_j$, donde p_j es la luz de índice j. Para ganar, ese valor debe ser 0, lo que equivale (sumando a_q a ambos lados y usando $a_q + a_q = 0$) a

$$\sum_{j : p_j \in V(q)} x_j \;=\; a_q, \tag{2}$$

donde se usó la simetría de la adyacencia para escribir la condición como $p_j \in V(q)$. Cada ecuación dice, entonces, que la cantidad de cambios de estado que recibe una luz debe ser impar si la luz está prendida y par si está apagada. El lado izquierdo suma las incógnitas de la luz y de sus vecinas, y el lado derecho es el estado inicial de la luz.

Reuniendo las n^2 ecuaciones se obtiene el sistema $A\mathbf{x} = \mathbf{b}$, donde $\mathbf{b} = (b_1, \ldots, b_{n^2})$ contiene los estados iniciales de las luces en el mismo orden que las incógnitas, y la matriz A, de tamaño $n^2 \times n^2$, está dada por

$$A_{ij} = \begin{cases} 1 & \text{si } p_j \in V(p_i), \\ 0 & \text{en otro caso.} \end{cases} \tag{3}$$

La columna j de A indica qué luces cambian al presionar p_j, y la fila i indica qué pulsaciones afectan a p_i. Por la simetría de la adyacencia, A es simétrica. La Figura 1 muestra el patrón de la matriz para $n = 4$.

Cabe aclarar que en la consigna, al describir la regla de asociación de las ecuaciones, se escribe la luz como $a_{(q-1)(r-1)}$; se entiende que se trata de una errata y que la ecuación i corresponde, igual que la incógnita x_i, a la luz $a_{(q+1)(r+1)}$, que es la que se usa en el ejemplo resuelto de la propia consigna.

Para el tablero 3×3 de la consigna,

$$T = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \qquad \begin{matrix} x_1 & x_2 & x_3 \\ x_4 & x_5 & x_6 \\ x_7 & x_8 & x_9 \end{matrix}$$

donde a la derecha se indica la incógnita que corresponde a cada posición. La Tabla 1 detalla la

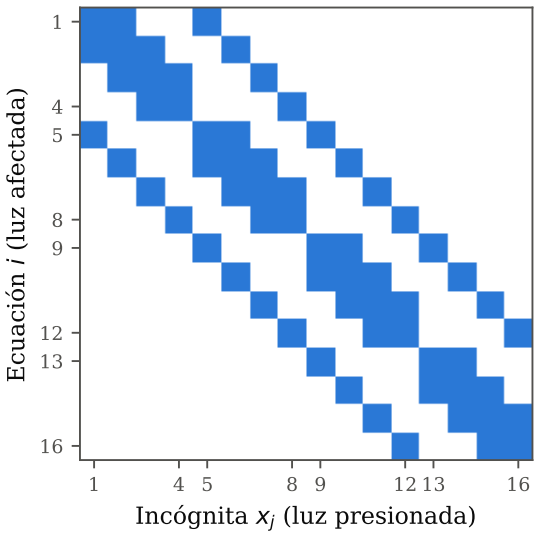


Figura 1. Matriz A del sistema para un tablero de 4×4. Cada casilla azul es un 1: la fila i es la ecuación de la luz i y la columna j es la incógnita de la luz j, con las luces numeradas por filas. La diagonal principal corresponde a la luz presionada, las diagonales vecinas a sus vecinas de la misma fila y las diagonales a distancia 4 a sus vecinas de la fila de arriba y de abajo.

construcción de cada una de las nueve ecuaciones. La luz a_{11} es una esquina, por lo que solo la afectan ella misma y sus dos vecinas a_{12} y a_{21}; como está apagada, la ecuación es $x_1 + x_2 + x_4 = 0$. Del mismo modo, a_{12} está en el borde superior y la afectan a_{11}, a_{12}, a_{13} y a_{22}; como está prendida, debe recibir una cantidad impar de cambios y la ecuación es $x_1 + x_2 + x_3 + x_5 = 1$. La luz central a_{22} tiene cuatro vecinas y por eso su ecuación es la única con cinco incógnitas. El resto de las ecuaciones se obtiene con el mismo razonamiento.

Tabla 1. Construcción de las ecuaciones del sistema para el tablero 3×3 de la consigna

Ec.	Luz	Ubicación	Luces que la afectan	Estado	Ecuación
1	a_{11}	esquina	a_{11}, a_{12}, a_{21}	0	$x_1 + x_2 + x_4 = 0$
2	a_{12}	borde	$a_{11}, a_{12}, a_{13}, a_{22}$	1	$x_1 + x_2 + x_3 + x_5 = 1$
3	a_{13}	esquina	a_{12}, a_{13}, a_{23}	0	$x_2 + x_3 + x_6 = 0$
4	a_{21}	borde	$a_{11}, a_{21}, a_{22}, a_{31}$	1	$x_1 + x_4 + x_5 + x_7 = 1$
5	a_{22}	centro	$a_{12}, a_{21}, a_{22}, a_{23}, a_{32}$	1	$x_2 + x_4 + x_5 + x_6 + x_8 = 1$
6	a_{23}	borde	$a_{13}, a_{22}, a_{23}, a_{33}$	0	$x_3 + x_5 + x_6 + x_9 = 0$
7	a_{31}	esquina	a_{21}, a_{31}, a_{32}	0	$x_4 + x_7 + x_8 = 0$
8	a_{32}	borde	$a_{22}, a_{31}, a_{32}, a_{33}$	0	$x_5 + x_7 + x_8 + x_9 = 0$
9	a_{33}	esquina	a_{23}, a_{32}, a_{33}	1	$x_6 + x_8 + x_9 = 1$

Nota. La columna *Estado* es el valor inicial de la luz en el tablero y es el término independiente de la ecuación. La luz a_{fc} corresponde a la incógnita $x_{3(f-1)+c}$.

3.3. Algoritmo de resolución

Para resolver el sistema se usa la escalerización de Gauss-Jordan sobre la matriz ampliada $[A \,|\, \mathbf{b}]$ con la suma de $\mathbb{F}_2$. La consigna indica que no es necesario el pivoteo y que la

única transformación a aplicar es $F_i \to F_i + F_j$. Para no suponer que la entrada en la posición de pivote vale siempre 1, el algoritmo comprueba ese valor y, si es 0, le suma a la fila del pivote la primera fila inferior que tenga un 1 en esa columna; así se usa únicamente la transformación indicada. La Sección 4.4 evalúa con qué frecuencia ocurre ese caso. El procedimiento se resume en el Algoritmo 1.

Algoritmo 1: Resolución de *Lights Out* por escalerización con suma binaria

Entrada : tablero T de $n \times n$ con entradas en $\{0,1\}$
Salida : vector $\mathbf{x} \in \mathbb{F}_2^{n^2}$ con las luces a presionar, o aviso de que no hay solución

Construir A según la Ecuación (3) y $\mathbf{b}$ recorriendo T por filas;
$M \leftarrow [A \,|\, \mathbf{b}]$; $f \leftarrow 1$; $P \leftarrow$ lista vacía de columnas pivote;
for $c \leftarrow 1$ **to** n^2 **do**
  if *ninguna fila* $k \geq f$ *tiene* $M_{kc} = 1$ **then**
    continuar con la siguiente columna (x_c es libre);
  if $M_{fc} = 0$ **then**
    Tomar la primera fila $k > f$ con $M_{kc} = 1$ y hacer $F_f \leftarrow F_f + F_k$;
  for *cada fila* $i \neq f$ *con* $M_{ic} = 1$ **do**
    $F_i \leftarrow F_i + F_f$; `// suma binaria`
  Agregar c a P; $f \leftarrow f + 1$;
if *alguna fila* $i \geq f$ *tiene término independiente 1* **then**
  return *no hay solución* ; `// fila` $0 = 1$
$\mathbf{x} \leftarrow \mathbf{0}$;
for *cada fila* i *con pivote en la columna* P_i **do**
  $x_{P_i} \leftarrow$ término independiente de F_i;
return $\mathbf{x}$

Como se eliminan los unos por encima y por debajo de cada pivote, al terminar cada fila no nula tiene un único 1 entre las columnas pivote, y la ecuación de esa fila determina el valor de su incógnita pivote una vez fijadas las libres. El algoritmo asigna 0 a las incógnitas libres, de modo que, cuando hay varias soluciones, devuelve una de ellas. Para analizar la cantidad de pulsaciones se implementó además una opción que recorre las $2^{\dim N(A)}$ soluciones, sumando a la solución anterior todas las combinaciones de una base del núcleo, y devuelve una de las que tienen menos unos. La base del núcleo se obtiene de la forma reducida: cada incógnita libre da un vector que vale 1 en ella, 0 en las demás libres y, en cada incógnita pivote, el valor que anula su ecuación.

3.4. Implementación

El método se implementó en Python 3 con las bibliotecas NumPy (Harris et al., 2020), para el manejo de matrices, y Matplotlib (Hunter, 2007), para las figuras. En el desarrollo y la redacción se utilizó un asistente de inteligencia artificial, según se detalla en la declaración al final del informe. Las filas se representan como vectores de enteros sin signo de 8 bits, y la operación $F_i \to F_i + F_f$ se calcula con el operador *o exclusivo* (XOR), que coincide con la suma

de $\mathbb{F}_2$ y opera sobre todas las columnas a la vez. El código fuente, las pruebas y las instrucciones para reproducir todos los resultados de este informe se encuentran en el repositorio https://github.com/GuillermoRivero98/lights-out. Los archivos principales son los siguientes:

- `lights_out.py`: módulo con el modelo y la función pedida, `resolver_lights_out`, que recibe una matriz $n \times n$ de ceros y unos y devuelve el vector solución, o lanza la excepción `SinSolucionError` si el tablero no tiene solución. Incluye también una simulación del juego (`presionar`, `aplicar_solucion`) que aplica las reglas directamente sobre el tablero, sin usar la matriz A.
- `test_lights_out.py`: pruebas automáticas que comparan el sistema construido con el de la consigna, verifican la jugada de ejemplo sobre a_{43}, comprueban las soluciones obtenidas y las comparan con una búsqueda exhaustiva.
- `experimentos.py`: genera todas las tablas y figuras de la Sección 4 y un archivo con los resultados numéricos.

3.5. Diseño de las pruebas y criterios de análisis

Se usaron cuatro conjuntos de casos. El primero son los ejemplos de la consigna: el tablero 3×3, cuyo sistema y solución se compararon con los dados, y el tablero 5×5 usado para ilustrar una jugada. El segundo es un tablero 3×3 tomado de una captura del juego en línea Grid Lights (Artbylogic, s.f.), que se reproduce en la Figura 2; se interpretó una luz verde como prendida y una roja como apagada. El tercero son tableros aleatorios: para cada n entre 1 y 10 se generaron 2000 tableros con entradas independientes que valen 0 o 1 con igual probabilidad. El cuarto son 200 tableros con solución para cada $n \in \{4,5,9\}$, generados como $\mathbf{b} = A\mathbf{z}$ con $\mathbf{z}$ aleatorio, que se usaron para comparar la cantidad de pulsaciones de las soluciones. Todos los tableros aleatorios se obtuvieron de un único generador de números pseudoaleatorios inicializado con la semilla 2026, de modo que los resultados son reproducibles.

Se aplicaron los siguientes criterios. Una solución se consideró correcta si, al aplicarla con la simulación del juego, el tablero queda completamente apagado; como la simulación no usa la matriz A, esta comprobación es independiente de la construcción del sistema. Además, en los tableros pequeños se compararon los resultados con una búsqueda exhaustiva: para los 512 tableros 3×3 se probaron los 512 conjuntos de pulsaciones posibles, y para $n = 4$ se obtuvieron los tableros con solución aplicando los 2^{16} conjuntos de pulsaciones al tablero apagado y se comprobó, en 300 tableros aleatorios, que la función informa que no hay solución exactamente cuando el tablero no está entre ellos. Para cada n se registraron el rango de A y la dimensión de su núcleo, y se comparó la fracción observada de tableros con solución con la fracción teórica $2^r/2^{n^2} = 2^{-\dim N(A)}$, que resulta de que la imagen de A tiene 2^r elementos. Por último, para evaluar la indicación de la consigna sobre el pivoteo, se escalerizó la matriz A en el orden natural de las ecuaciones tomando cada fila tal como está, y se registró el primer paso en que el elemento de la diagonal es 0. También se registró cuántas veces el algoritmo tuvo que corregir

el pivote.

4. Resultados

4.1. Ejemplo de la consigna

La matriz A y el vector $\mathbf{b}$ construidos por el programa para el tablero 3×3 coinciden ecuación por ecuación con el sistema de la consigna, y la función devolvió el vector $\mathbf{x} = (0,0,0,0,1,0,0,0,1)$, es decir, $x_5 = x_9 = 1$, que corresponde a presionar las luces a_{22} y a_{33}. La Tabla 2 muestra la sustitución de esta solución en cada ecuación: en todos los casos el lado izquierdo coincide con el término independiente. En las ecuaciones 6 y 8 aparecen a la vez x_5 y x_9, y la suma $1+1=0$ refleja que las luces a_{23} y a_{32} cambian dos veces de estado.

Tabla 2. Verificación de la solución $x_5 = x_9 = 1$, $x_i = 0$ en las demás incógnitas, para el tablero 3×3

Ec.	Ecuación	Sustitución	Resultado	Término independiente
1	$x_1+x_2+x_4$	$0+0+0$	0	0
2	$x_1+x_2+x_3+x_5$	$0+0+0+1$	1	1
3	$x_2+x_3+x_6$	$0+0+0$	0	0
4	$x_1+x_4+x_5+x_7$	$0+0+1+0$	1	1
5	$x_2+x_4+x_5+x_6+x_8$	$0+0+1+0+0$	1	1
6	$x_3+x_5+x_6+x_9$	$0+1+0+1$	0	0
7	$x_4+x_7+x_8$	$0+0+0$	0	0
8	$x_5+x_7+x_8+x_9$	$1+0+0+1$	0	0
9	$x_6+x_8+x_9$	$0+0+1$	1	1

Nota. Las sumas se calculan en $\{0,1\}$ con $1+1=0$.

Para este tablero la escalerización no dejó incógnitas libres (rango 9). La primera fila de la Figura 3 muestra el tablero inicial, las luces presionadas y el tablero final obtenido con la simulación.

4.2. Captura del juego en línea y tablero 5×5 de la consigna

Para el tablero de la captura de Grid Lights,

$$T = \begin{pmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix},$$

la función devolvió $\mathbf{x} = (0,0,1,0,0,0,0,0,0)$, es decir, presionar únicamente la luz a_{13}. La Figura 2 muestra la captura original y la segunda fila de la Figura 3, el resultado.

La simulación de la jugada sobre a_{43} en el tablero 5×5 de la consigna reprodujo exactamente el tablero que se muestra allí. Sin embargo, el sistema asociado a ese tablero resultó incompatible: la escalerización produjo una fila de la forma $0 = 1$. Para $n = 5$ el núcleo de A

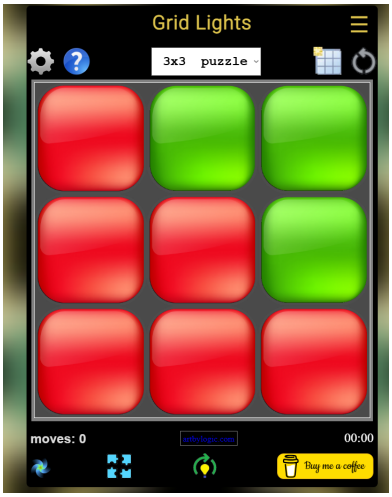


Figura 2. Captura del juego en línea Grid Lights en modo 3×3, usada como caso de prueba (Artbylogic, s.f.). Se interpretan las luces verdes como prendidas (1) y las rojas como apagadas (0), lo que da el tablero de la Sección 4.2.

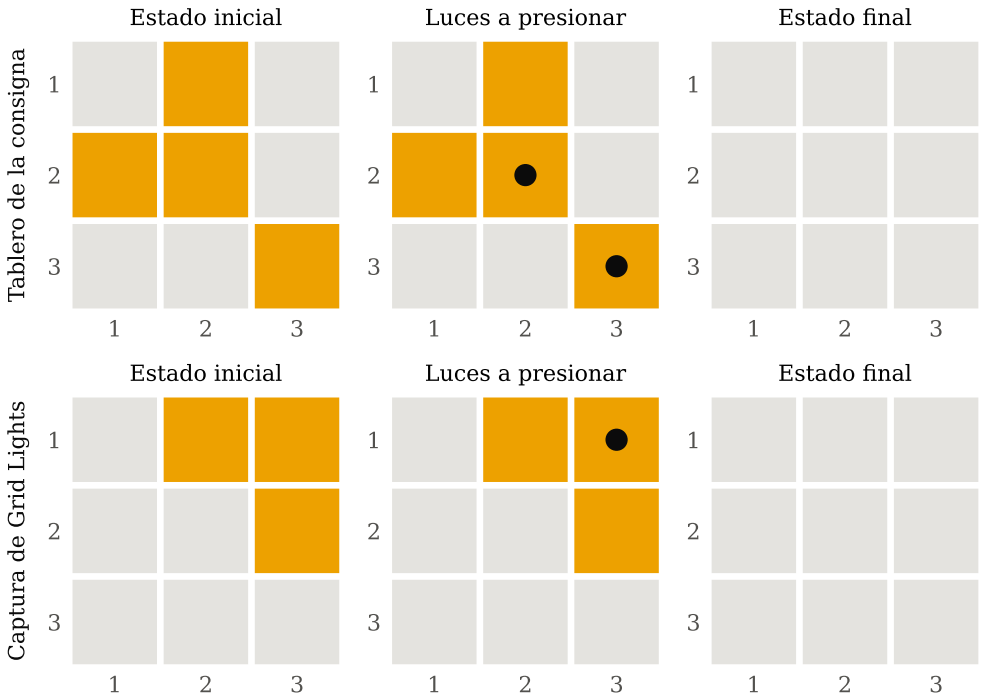


Figura 3. Resolución de los dos tableros 3×3: arriba, el ejemplo de la consigna; abajo, la captura del juego Grid Lights. Las casillas anaranjadas son luces prendidas y las grises, apagadas. La columna central marca con un punto negro las luces que indica presionar la función `resolver_lights_out`, y la columna derecha es el tablero que resulta de presionarlas según las reglas del juego.

tiene dimensión 2 y una base es

$$\mathbf{z}_1 = \begin{pmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{pmatrix}, \qquad \mathbf{z}_2 = \begin{pmatrix} 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 \end{pmatrix},$$

escritos como tableros. El producto (con la suma de $\mathbb{F}_2$) del vector $\mathbf{b}$ de ese tablero con $\mathbf{z}_1$ es 1 y con $\mathbf{z}_2$ es 0.

4.3. Estructura del sistema según el tamaño del tablero

La Tabla 3 resume, para cada n entre 1 y 10, el rango de A, la dimensión de su núcleo, la fracción de tableros con solución esperada y la observada en los 2000 tableros aleatorios, y la cantidad de soluciones que apagaron el tablero en la simulación. La matriz tiene rango completo para $n \in \{1,2,3,6,7,8,10\}$, y en esos tamaños todos los tableros generados tuvieron solución. Para $n = 4$, $n = 5$ y $n = 9$ el núcleo tiene dimensión 4, 2 y 8, respectivamente, y la proporción de tableros con solución fue de 0,0615, 0,2475 y 0,0035, frente a los valores teóricos 0,0625, 0,25 y 0,0039. En todos los tamaños, el 100 % de las soluciones devueltas por la función apagó el tablero al aplicarse con la simulación. La Figura 4 representa la comparación entre ambas fracciones.

Tabla 3. Rango y núcleo de la matriz del sistema, fracción de tableros aleatorios con solución, soluciones verificadas por simulación y comportamiento del pivote, para n entre 1 y 10

n	Incógnitas	Rango	$\dim N(A)$	Fracción con solución		Soluciones	Paso con	Correcciones
	(n^2)	de A		Teórica	Observada	verificadas	pivote nulo	del pivote
1	1	1	0	1,0000	1,0000	2000/2000	—	0
2	4	4	0	1,0000	1,0000	2000/2000	2	1
3	9	9	0	1,0000	1,0000	2000/2000	2	3
4	16	12	4	0,0625	0,0615	123/123	2	4
5	25	23	2	0,2500	0,2475	495/495	2	9
6	36	36	0	1,0000	1,0000	2000/2000	2	14
7	49	49	0	1,0000	1,0000	2000/2000	2	20
8	64	64	0	1,0000	1,0000	2000/2000	2	26
9	81	73	8	0,0039	0,0035	7/7	2	30
10	100	100	0	1,0000	1,0000	2000/2000	2	41

Nota. La fracción teórica es $2^{-\dim N(A)}$. La fracción observada se calculó sobre 2000 tableros aleatorios por tamaño (semilla 2026). *Soluciones verificadas* indica cuántas de las soluciones obtenidas apagaron el tablero en la simulación, sobre el total de tableros con solución. *Paso con pivote nulo* es el primer paso de la escalerización, tomando cada fila tal como está, en el que la entrada diagonal es 0; un guion indica que ese caso no se presenta. *Correcciones del pivote* es la cantidad de pasos en que el algoritmo tuvo que sumar a la fila del pivote una fila inferior.

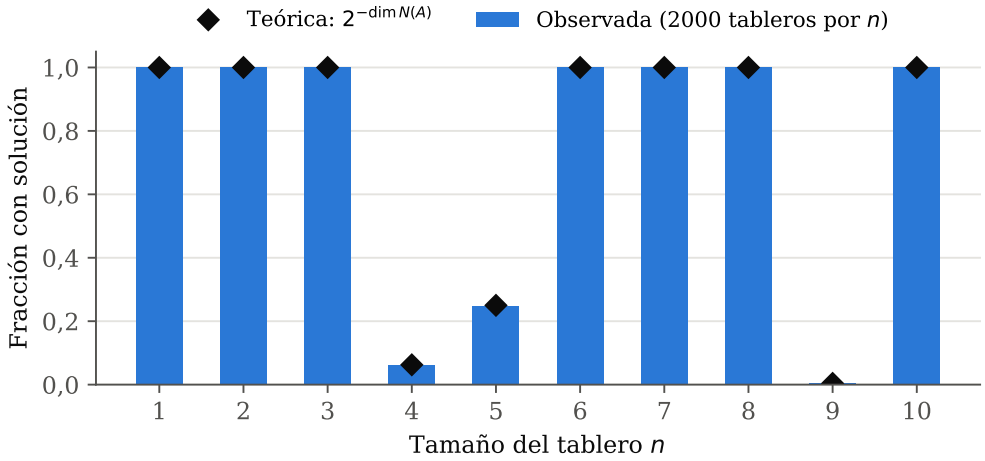


Figura 4. Fracción de tableros $n \times n$ que tienen solución. Las barras muestran la proporción observada en 2000 tableros aleatorios por tamaño y los rombos, el valor teórico $2^{-\dim N(A)}$. Para $n = 9$ la proporción es 0,0035, por lo que la barra no es visible a esta escala.

4.4. Entradas nulas en la posición de pivote

Al escalerizar A en el orden natural de las ecuaciones tomando cada fila tal como está, la entrada diagonal del segundo paso fue 0 para todos los tamaños entre $n = 2$ y $n = 10$ (penúltima columna de la Tabla 3). Para $n = 3$, las dos primeras filas de A son

$$F_1 = (1,1,0,1,0,0,0,0,0), \qquad F_2 = (1,1,1,0,1,0,0,0,0),$$

y la eliminación de la primera columna, $F_2 \to F_2 + F_1$, da $F_2 = (0,0,1,1,1,0,0,0,0)$, cuya segunda entrada es 0, mientras que la fila $F_3 = (0,1,1,0,0,1,0,0,0)$ tiene un 1 en esa columna. La transformación $F_2 \to F_2 + F_3$ da $F_2 = (0,1,0,1,1,1,0,0,0)$, con un 1 en la posición de pivote. Para n entre 2 y 10, el algoritmo tuvo que corregir el pivote de esta forma entre 1 y 41 veces según el tamaño (última columna de la Tabla 3).

4.5. Cantidad de pulsaciones

La Tabla 4 compara, para los tamaños con más de una solución por tablero, la cantidad media de luces que presiona la solución con incógnitas libres en 0 y la de la solución con menos pulsaciones. En los tres tamaños la solución mínima requirió en promedio menos pulsaciones: 4,225 frente a 6,195 para $n = 4$, 9,915 frente a 11,225 para $n = 5$ y 27,790 frente a 36,640 para $n = 9$.

5. Discusión

Los resultados confirman que el modelo lineal describe correctamente el juego. El sistema generado por el programa coincide con el de la consigna, la solución $x_5 = x_9 = 1$ satisface las nueve ecuaciones y todas las soluciones obtenidas para tableros aleatorios apagaron el ta-

Tabla 4. Cantidad media de luces presionadas por la solución con incógnitas libres en 0 y por la solución mínima

n	$\dim N(A)$	Tableros	Pulsaciones medias	
			Libres en 0	Mínima
4	4	200	6,195	4,225
5	2	200	11,225	9,915
9	8	200	36,640	27,790

Nota. Tableros con solución generados como $A\mathbf{z}$ para $\mathbf{z}$ aleatorio (semilla 2026). Cada tablero admite $2^{\dim N(A)}$ soluciones.

blero en una simulación que no depende de la matriz A. Como para $n = 3$ la matriz tiene rango 9, la solución encontrada para el tablero de la consigna y para el de la captura es, en cada caso, la única, lo que coincide con la búsqueda exhaustiva sobre todos los tableros 3×3. Esto último es relevante porque un error en la construcción de A (por ejemplo, en la numeración de las luces o en el tratamiento de los bordes) produciría soluciones que satisfacen el sistema pero no ganan el juego, y la comparación con la simulación lo habría detectado.

El hallazgo más importante es que la existencia de solución depende del tamaño del tablero. Cuando A es invertible, cada tablero tiene exactamente una solución; cuando no lo es, la imagen de A es un subespacio propio de $\mathbb{F}_2^{n^2}$ y solo los tableros que pertenecen a ella pueden apagarse. Para $n = 5$ el rango es 23, lo que coincide con lo reportado por Anderson y Feil (1998), y solo uno de cada cuatro tableros tiene solución. Las frecuencias observadas en la Tabla 3 difieren de las teóricas en a lo sumo 0,0025, una diferencia menor que el error estándar de una proporción estimada con 2000 tableros (aproximadamente 0,01 para $n = 5$), por lo que el acuerdo entre teoría y experimento es el esperable. El caso $n = 9$ muestra que el tamaño del núcleo no crece de forma regular con n: allí solo 7 de los 2000 tableros tuvieron solución, mientras que para $n = 10$ la tuvieron todos.

El tablero 5×5 de la consigna ilustra este fenómeno. La consigna lo usa solamente para mostrar una jugada, pero la incompatibilidad de su sistema indica que ese estado no puede apagarse. Como A es simétrica, su imagen coincide con el conjunto de vectores ortogonales (con la suma de $\mathbb{F}_2$) a todos los vectores de su núcleo (Anderson y Feil, 1998), y por eso basta comprobar dos productos: el producto con $\mathbf{z}_1$ es 1, lo que muestra que el tablero no está en la imagen. Este criterio tiene una interpretación concreta. Presionar las luces marcadas en $\mathbf{z}_1$ no cambia ningún tablero, y el producto $\mathbf{z}_1 \cdot \mathbf{b}$ cuenta, módulo 2, las luces prendidas en las posiciones marcadas por $\mathbf{z}_1$. Toda pulsación cambia una cantidad par de esas luces, por lo que la paridad se conserva durante todo el juego; si al comienzo es impar, nunca se llega al tablero apagado, cuya paridad es par. Además, como la jugada sobre a_{43} suma a $\mathbf{b}$ una columna de A, que pertenece a la imagen, el tablero resultante tampoco tiene solución.

En cuanto al método, los resultados de la Sección 4.4 muestran que la entrada en la posición de pivote no siempre vale 1. La razón es estructural: para todo $n \geq 2$ las luces 1 y 2 son vecinas, por lo que las dos primeras filas de A tienen un 1 en las columnas 1 y 2, y al sumar

F_1 a F_2 ambas entradas se anulan. Una implementación que tomara cada fila tal como está y asumiera que el pivote vale 1 fallaría desde el segundo paso. Sin embargo, esto no contradice la indicación de la consigna de usar únicamente $F_i \to F_i + F_j$: basta con sumar a la fila del pivote una fila inferior que tenga un 1 en esa columna, como hace el Algoritmo 1, sin necesidad de intercambiar filas. En ese sentido, lo que la consigna llama "no hacer pivoteo" es posible, siempre que se busque la fila adecuada antes de eliminar. Además, a diferencia de lo que ocurre con números reales, en $\mathbb{F}_2$ no hace falta elegir el pivote de mayor valor absoluto para reducir errores de redondeo, porque todas las operaciones son exactas.

Cuando hay varias soluciones, la elección de las incógnitas libres afecta la cantidad de pulsaciones, como muestra la Tabla 4. Asignar 0 a las incógnitas libres es una elección cómoda, pero no produce la solución más corta: en $n = 4$ la solución mínima requiere en promedio alrededor de un tercio menos de pulsaciones. Obtener la mínima exige recorrer las $2^{\dim N(A)}$ soluciones, lo que es posible para los tamaños estudiados pero crece exponencialmente con la dimensión del núcleo.

El trabajo tiene algunas limitaciones. La escalerización realiza del orden de n^4 operaciones de fila (hasta n^2 por cada una de las n^2 columnas), cada una sobre unas n^2 entradas, lo que da del orden de n^6 operaciones sobre bits; esto es suficiente para los tamaños usados en el juego, pero sería lento para tableros mucho más grandes, en los que convendría aprovechar que A es dispersa. Los resultados empíricos se limitan a $n \leq 10$ y a una única semilla. Por último, para la captura de Grid Lights se supuso que la luz verde está prendida; si se usara la convención opuesta, el tablero inicial sería el complemento y la solución cambiaría, aunque el método sería el mismo.

6. Conclusiones

Se mostró que el juego *Lights Out* admite un modelo lineal exacto. Como presionar dos veces una luz no tiene efecto y el orden de las pulsaciones no altera el resultado, cada solución se representa con un vector de $\mathbb{F}_2^{n^2}$, y apagar el tablero equivale a resolver el sistema $A\mathbf{x} = \mathbf{b}$, donde A codifica los vecindarios de las luces y $\mathbf{b}$ el estado inicial. Para el ejemplo 3×3 de la consigna se justificaron las nueve ecuaciones y se verificó que presionar a_{22} y a_{33} es su única solución.

La función implementada en Python resuelve el sistema mediante escalerización de Gauss-Jordan con suma binaria y resolvió correctamente todos los tableros con solución que se le presentaron. Para que funcione en todos los casos fue necesario contemplar que la entrada en la posición de pivote puede ser 0, lo que se resolvió sumando una fila inferior, dentro de la única transformación indicada en la consigna, y detectar los sistemas incompatibles.

En cuanto al cuarto objetivo, se concluye que todo tablero tiene solución única cuando la matriz del sistema es invertible, lo que ocurre para $n \in \{1, 2, 3, 6, 7, 8, 10\}$ dentro de los tamaños estudiados. Para $n \in \{4, 5, 9\}$ solo una fracción $2^{-\dim N(A)}$ de los tableros tiene solución, y cada uno de ellos admite $2^{\dim N(A)}$ soluciones distintas. El mismo análisis permite decidir antes de

jugar si un tablero tiene solución, comprobando su paridad respecto de los vectores del núcleo, como se mostró con el tablero 5×5 de la consigna.

Como líneas de trabajo futuro, se propone aprovechar la estructura dispersa de A para resolver tableros de mayor tamaño, estudiar con mayor generalidad para qué valores de n la matriz es invertible y extender el modelo a variantes del juego, como tableros rectangulares o luces con más de dos estados, en las que la suma se calcula módulo un número mayor.

Declaración de uso de inteligencia artificial

En la elaboración de este trabajo se utilizó el asistente de inteligencia artificial Claude (Anthropic, 2026). La herramienta se usó en las siguientes tareas:

- análisis de la consigna, de la guía de informes y de la rúbrica de corrección;
- desarrollo del modelo matemático y redacción de las demostraciones de las Proposiciones 1 a 3;
- implementación del código en Python (`lights_out.py`, `test_lights_out.py` y `experimentos.py`) y generación de las tablas y figuras;
- redacción del informe en LaTeX;
- revisión del informe y del código contra la consigna y la rúbrica, incluida una verificación independiente de los resultados numéricos.

Los integrantes del grupo definieron el alcance del trabajo, aportaron los casos de prueba (incluida la captura de Grid Lights y la convención de colores para interpretarla), revisaron la coherencia de todo el documento y comprobaron a mano el desarrollo y los resultados. Los integrantes asumen la responsabilidad por el contenido final del informe.

Referencias

Anderson, M., y Feil, T. (1998). Turning lights out with linear algebra. *Mathematics Magazine, 71*(4), 300–303. https://doi.org/10.1080/0025570X.1998.11996658

Anthropic. (2026). *Claude* (versión Opus 5.5) [Modelo de lenguaje de gran escala]. https://claude.ai

Artbylogic. (s.f.). *Grid Lights* [Juego en línea]. https://artbylogic.com

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., Fernández del Río, J., Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature, 585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hoffman, K., y Kunze, R. (1971). *Linear algebra* (2.ª ed.). Prentice-Hall.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Lay, D. C., Lay, S. R., y McDonald, J. J. (2016). *Linear algebra and its applications* (5.ª ed.). Pearson.

Lidl, R., y Niederreiter, H. (1997). *Finite fields* (2.ª ed.). Cambridge University Press.

Sutner, K. (1989). Linear cellular automata and the Garden-of-Eden. *The Mathematical Intelligencer, 11*(2), 49–53.

Universidad Católica del Uruguay. (s.f.). *Proyecto Lights Out!* [Consigna del curso Álgebra Aplicada]. Facultad de Ingeniería y Tecnologías.